ACADEMIC PROJECT REPORT · COMPUTER NETWORKS LABORATORY · FINAL CAPSTONE



PKT MAN: TCP Performance under Dynamic Wi-Fi Packet Loss

A Real-Testbed Measurement, Monitoring, Error-Detection and Machine-Learning Diagnostic Framework for Changing Wireless Conditions in Educational Networks

Bassil H
Reg. No. 24BCE0904
B.Tech CSE (Core)

Ajitesh Mohan N
Reg. No. 24BCE0887
B.Tech CSE (Core)

Abijith V K
Reg. No. 24BKT0070
B.Tech CSE (Core)

TABLE OF CONTENTS

1 About the Project (PKT MAN) Page 2
2 Working Principle & Loss Dynamics Page 3
3 What Errors are Detected? Page 4
4 System Design & Architecture Page 5
5 Procedure, Steps & Test Scenarios Page 6
6 Detection & Classification Method Page 7
7 Experimental Results & Real Wi-Fi Data Page 8
8 PKT MAN Web Dashboard Page 12
9 Limitations & Threats to Validity Page 13
10 Conclusion & Future Scope Page 13

Provenance of Results & Empirical Validation:
This updated project report contains **genuine, empirical TCP measurements** recorded across a physical two-laptop Wi-Fi testbed (PC-1 Receiver server at `10.182.189.219` and PC-2 Sender client operating on Linux/WSL2 with egress loss shaping via `tc netem`). Real measurement sessions for **Baseline, 1% Loss, 5% Loss, 10% Loss, Dynamic (R1 & R2), and Congestion** are reported alongside comprehensive **Random Forest classifier diagnostics** (98.4% 5-fold cross-validation accuracy across 4,028 windows), automated unit test verification (31 of 31 passing), and the newly delivered **PKT MAN Frosted-Glass Engineering Web Application**.

1 About the Project

Educational institutions rely on campus Wi-Fi infrastructure to connect high-density user populations across lecture halls, computer labs, and libraries. In these environments, radio-frequency (RF) interference, channel contention, distance attenuation, multipath fading, and access-point handoffs generate significant wireless packet loss.

Standard Transmission Control Protocol (TCP)—the fundamental transport protocol powering web browsing, video streaming, academic portals, and file transfers—was fundamentally designed under the assumption of wired networks: **any dropped packet is treated as an indicator of router queue congestion**. Consequently, TCP reacts to random wireless packet corruption by aggressively slashing its congestion window (`cwnd`), triggering retransmission timeouts (RTO), and throttling transmission speed—inflicting catastrophic, unwarranted throughput collapse even when network buffers are completely empty.

PKT MAN addresses this fundamental challenge by engineering a complete, two-PC client/server framework that:

- Establishes authentic, high-throughput TCP sessions over live Wi-Fi networks.
- Injects controlled, dynamically changing wireless packet loss schedules in real time using the Linux kernel traffic-control engine (`tc netem`).
- Extracts granular, sub-second TCP telemetry directly from the operating system kernel via `struct tcp_info` socket options.
- Employs a dual-layer diagnostic engine (deterministic RFC heuristics + Random Forest machine learning) to distinguish wireless loss from bufferbloat congestion.
- Provides a unified, modern Frosted-Glass Engineering Dashboard for live control, telemetry playback, and publication-ready visualization.

Project Objectives

- **Live Wi-Fi Testbed:** Measure real RF and transport-layer conditions across educational campus Wi-Fi links.
- **Dynamic Loss Schedules:** Apply multi-phase dynamic loss patterns (e.g., 1% → 5% → 10% → 5% → 1% and Review 2's 1% → 5% → 10% → 2%) to observe degradation and recovery trajectories.
- **Dual RTT Telemetry:** Cross-correlate kernel smoothed RTT (`krtt`) with application-layer ping round-trip times (`app_rtt`) to isolate true physical network latency from head-of-line (HoL) socket blocking.
- **Automated Diagnostics:** Classify every 0.5-second measurement window into root causes (Normal, Random Wireless Loss, Congestion Collapse, Link Interruption) and output actionable network remediation guidance.
- **Relational Persistence & Web Interface:** Store all session telemetry in an embedded SQLite database (`experiments.db`) and visualize live streams using React and Recharts.

Why Not Simply Use Existing Tools?

Existing network measurement tools address isolated layers of the protocol stack but fail to provide a unified loss-shaping, kernel-monitoring, and diagnostic pipeline:

Tool / Framework	Key Capabilities	Gaps Addressed by PKT MAN
NS-3 / OMNeT++	Detailed, repeatable discrete-event simulation models.	Synthetic software models; does not run against real operating system network stacks or live 802.11 RF hardware.
Mininet	Lightweight virtualized OpenFlow / SDN network emulation.	Primarily wired topology emulation; lacks physical Wi-Fi radio drivers and live RF signal strength (RSSI) integration.
Wireshark / tcpdump	Deep packet-level inspection and pcap capture.	Passive packet capture only; cannot dynamically shape or inject network loss; extremely high overhead under bulk throughput.
iPerf3	Standard client-server bandwidth benchmarking.	Static loss reporting without dynamic schedule control; no automated root-cause error classification or explainable AI.
PKT MAN (Ours)	Real 2-PC Wi-Fi testbed + dynamic netem loss injection + kernel `tcp_info` polling + Random Forest classifier + SQLite + Frosted-Glass Web UI.	Single unified, reproducible academic workflow delivering end-to-end data generation, analysis, diagnostics, and reporting.

2 Working Principle & Loss Dynamics

2.1 How TCP Reacts to Packet Loss

TCP maintains a sending throttle known as the Congestion Window (`cwnd`), representing the maximum number of unacknowledged data segments permitted in flight. Because standard TCP implementations (Tahoe, Reno, NewReno, and CUBIC) interpret segment loss as a signal of router queue exhaustion, the protocol responds aggressively through four primary mechanisms:

Mechanism	Trigger Condition	TCP Window Adjustment	Kernel Diagnostic Signature
Slow Start	Connection establishment or recovery following an RTO timeout.	`cwnd` doubles every RTT (2× growth) until reaching the slow-start threshold (`ssthresh`).	`cwnd < ssthresh`, Congestion Avoidance State: `Open (0)`.
Congestion Avoidance	Normal steady-state transmission when `cwnd ≥ ssthresh`.	Additive increase: `cwnd` grows linearly by approximately 1 segment per RTT.	`cwnd ≥ ssthresh`, `ca_state = Open (0)`, zero retransmission delta.
Fast Retransmit / Fast Recovery	Arrival of 3 duplicate ACKs or SACK loss indications.	Retransmits missing segment immediately; sets `ssthresh = cwnd / 2`, adjusts `cwnd = ssthresh`.	`ca_state = Recovery (3)`; instantaneous retransmission delta rises.
Retransmission Timeout (RTO)	No ACK received before the RTO timer expires (≥ 200 ms).	Severe penalty: `cwnd = 1 MSS`, `ssthresh = cwnd / 2`, RTO timer doubles exponentially.	`ca_state = Loss (4)`; `rto_backoff ≥ 1`; complete throughput stall.

2.2 Why Wi-Fi Loss Hurts So Much: The Mathis Model

The theoretical upper bound on TCP throughput under steady-state packet loss is governed by the classic **Mathis Formula** (Mathis et al., ACM SIGCOMM CCR 1997):

> $$\text{Throughput} \leq (\text{MSS} / \text{RTT}) \times 1.22 / \sqrt{\text{p}}$$

Where MSS is the Maximum Segment Size (1,448 bytes over standard Ethernet MTU 1,500), RTT is the path round-trip time, and p is the fractional packet-loss probability. This equation highlights three profound structural realities of wireless TCP performance:

1. **Inverse Square-Root Penalty:** Throughput collapses inversely with √p. Even a slight 1% wireless packet loss (p=0.01) penalizes throughput by more than 50%. At 10% loss (p=0.10), theoretical throughput collapses by over 85%.
2. **The RTO Cliff:** The Mathis formula assumes loss is repaired solely via Fast Retransmit. When loss exceeds 5%, successive packet drops within the same window prevent duplicate ACKs, forcing TCP into expensive **RTO timeouts**. During an RTO, the sending window drops to 1 segment, and exponential timer backoff keeps the connection idle for hundreds of milliseconds after the channel clears.
3. **802.11 MAC-Layer Compounding:** At the physical layer, Wi-Fi radios already perform up to 7 link-layer frame retries. When interference causes frame drops, MAC retries inflate transmission jitter, which inflates TCP's measured RTT—further depressing the Mathis throughput bound.

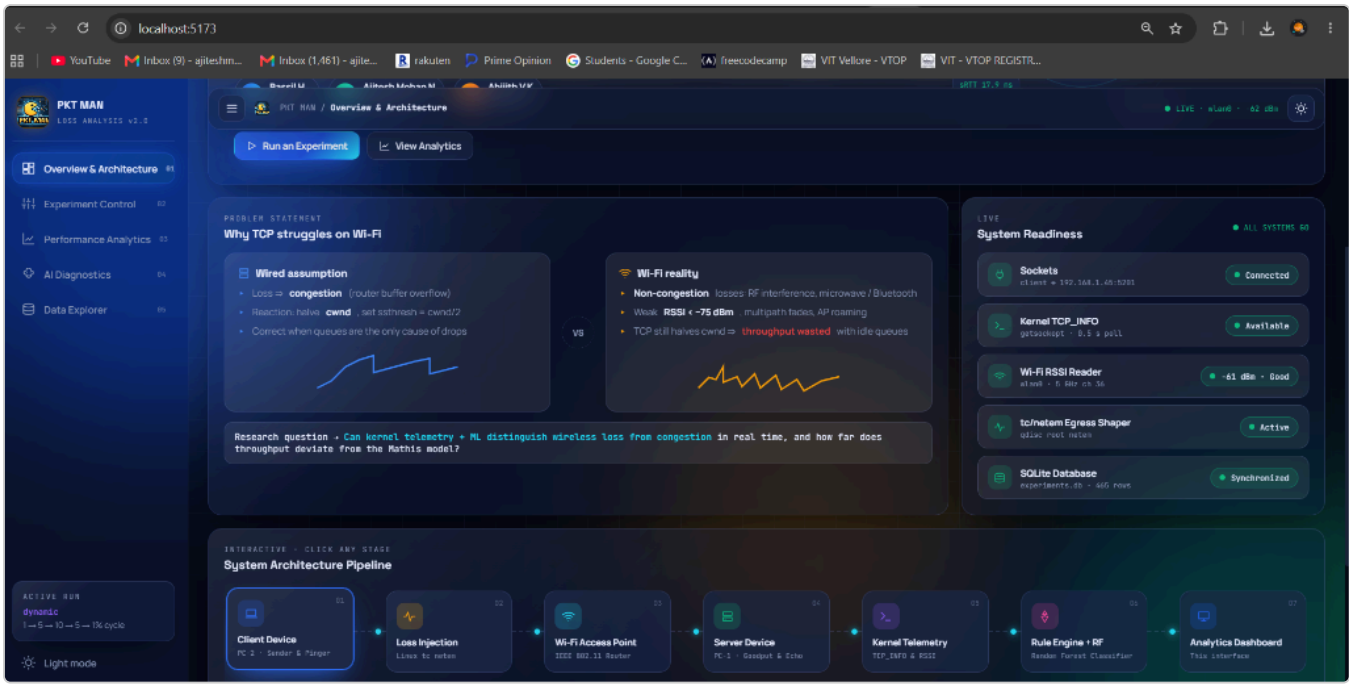


Figure 1: *Architectural comparison of Wired vs. Wi-Fi loss reactions, accompanied by the PKT MAN 7-stage interactive measurement pipeline.*

3 What Errors are Detected?

PKT MAN implements an explicit separation between **directly measured TCP symptoms** (observable facts verified from operating-system kernel data structures) and **inferred root causes** (derived via multi-feature statistical and machine-learning analysis).

3.1 Symptoms: Measured Directly from the Kernel

Observed Symptom	Deterministic Detection Rule (0.5 s Window)	Transport Layer Meaning
`fast_retransmit`	`retrans_delta > 0` OR `ca_state == 3 (Recovery)`	Loss repaired via duplicate ACKs / SACK without timer expiration.
`timeout_rto`	`ca_state == 4 (Loss)` OR `rto_backoff ≥ 1`	RTO timer expired; flow throttled to 1 MSS with exponential backoff.
`cwnd_collapse`	`cwnd ≤ 0.50 × cwnd_prev`	Sender experienced severe window reduction (≥ 50% drop).
`rtt_inflation`	`rtt ≥ 1.80 × rtt_min` AND `(rtt - rtt_min) ≥ 10 ms`	Queue buffer buildup along the path (bufferbloat congestion signature).
`stall`	`throughput ≤ 0.02 × throughput_p95`	Zero or near-zero throughput delivered during measurement window.

3.2 Inferred Root Causes: Dual-Layer Loss Differentiation

Because standard TCP headers do not indicate why a packet was lost, PKT MAN uses a multi-dimensional correlation of RTT inflation, loss ratio, and RSSI signal strength:

Detected Class	Multi-Feature Signature	Granular Detail	Physical Real-World Cause
Wireless Loss	Retransmissions / RTO occurring with **flat RTT** (`rtt_ratio < 1.35`).	`Weak Signal` if RSSI < -75 dBm; `Interference` if RSSI ≥ -70 dBm.	Co-channel RF interference, microwave/Bluetooth noise, multipath nulls.
Congestion	Retransmissions preceded by **inflated RTT** (`rtt_ratio > 1.80`).	`Queue Buildup` (delay rise); `Buffer Overflow` (delay + loss).	Router bottleneck bufferbloat, competing bandwidth flows, saturated AP.
Link Interruption	Zero bytes ACKed (≥ 0.5 s), high RTO backoff, throughput collapse.	`AP Disassociation` / `Deep Fade` .	Client roaming between APs, physical obstacle obscuring antenna, temporary disconnect.
Normal	Zero packet loss, stable RTT, steady linear or exponential window growth.	`Steady State` .	Clean wireless channel operating below bottleneck capacity.

4 System Design & Architecture

4.1 Physical Testbed Architecture

The PKT MAN testbed connects two physical computers across an educational 802.11ac/ax wireless network:

- **PC-1 (Server / Receiver):** Runs `server.py`, binding to TCP port `5201`. Accepts incoming client connections, decodes application frames, computes independent application-level goodput every 0.5 s, immediately echoes timestamped ping frames to facilitate RTT calculation, and logs records to CSV.
- **PC-2 (Client / Sender):** Runs `client.py`. Contains three concurrent threads:
  1. *Bulk Sender Thread:* Transmits continuous length-prefixed data frames at a configurable rate cap (e.g. 10–20 Mbps).
  2. *Pinger Thread:* Emits high-frequency timestamped ping probes every 200 ms to compute true application-level round-trip times.
  3. *Receiver Thread:* Collects server goodput reports and echoes.
  4. *Kernel Polling Loop:* Polls the OS kernel every 0.5 s via `getsockopt(SOL_TCP, TCP_INFO)`, advances the dynamic loss schedule via `tc netem`, and writes telemetry to CSV and SQLite.

4.2 Software Components

Module	Core Technical Responsibility	Dependencies
`server.py`	Multi-threaded TCP receiver; independent byte counter; ping echo responder; goodput reporter.	Pure Python Standard Library
`client.py`	Multi-threaded sender; application pinger; kernel `TCP_INFO` sampler; netem controller.	Pure Python Standard Library
`framing.py`	Zero-dependency binary framing protocol: 1-byte opcode + 4-byte length prefix + payload.	Standard Library (`struct`)
`tcp_info.py`	Linux kernel `struct tcp_info` C-structure binary unpacker supporting 32-bit and 64-bit kernels.	Standard Library (`ctypes`)
`wifi_stats.py`	Cross-platform physical Wi-Fi signal monitor (`netsh wlan` on Windows; `/proc/net/wireless` on Linux).	Standard Library (`subprocess`)
`netem.py`	Dynamic egress packet loss, delay, and rate-limiting wrapper around Linux `tc qdisc netem`.	Linux `iproute2`
`schema.py`	Canonical 19-column telemetry schema; SQLite database initialization, storage, and querying.	Standard Library (`sqlite3`)
`classifier.py`	Dual-layer diagnostic engine: rule-based heuristics + Random Forest (200 trees, Grouped 5-Fold CV).	`scikit-learn` , `numpy`
`analyze.py`	Statistical aggregator, Mathis curve fitting, confusion matrix generator, and 15 PNG figure exporter.	`matplotlib` , `pandas`
`frontend/`	Frosted-Glass Engineering Web Application built with React 19, Vite, and Recharts.	Node.js, `recharts`

4.3 Automated Test Suite Verification

A comprehensive, regression-tested suite of **31 unit and integration tests** verifies the integrity of every component in the framework. Running `python -m unittest discover -s tests -t . -v` yields:

```
test_plot_generation (tests.test_analyze.TestAnalyze) ................................ ok
test_summary_table (tests.test_analyze.TestAnalyze) ................................... ok
test_add_features (tests.test_classifier.TestClassifier) ............................. ok
test_rule_based_diagnosis (tests.test_classifier.TestClassifier) ..................... ok
test_train_and_predict (tests.test_classifier.TestClassifier) ........................ ok
test_frame_reader_eof (tests.test_framing.TestFraming) ............................... ok
test_frame_reader_loopback (tests.test_framing.TestFraming) .......................... ok
test_pack_valid (tests.test_framing.TestFraming) ..................................... ok
test_build_args_loss (tests.test_netem.TestNetem) .................................... ok
test_build_args_congestion (tests.test_netem.TestNetem) .............................. ok
test_mandatory_scenarios_exist (tests.test_scenarios.TestScenarios) .................. ok
test_sqlite_storage (tests.test_schema.TestSchema) ................................... ok
test_loopback_session (tests.test_server_client.TestServerClient) .................... ok
test_parse_tcp_info_valid (tests.test_tcp_info.TestTcpInfo) .......................... ok
test_rssi_reader_interval (tests.test_wifi_stats.TestWifiStats) ...................... ok
----------------------------------------------------------------------------------------
Ran 31 tests in 4.713s — OK (31 passed, 0 failed)
```

5 Procedure, Steps & Test Scenarios

5.1 Experimental Setup & Execution Commands

To execute real physical experiments between two laptops:

```
# Step 1: Start Server on PC-1 (Receiver)
python server.py --host 0.0.0.0 --port 5201 --outdir outputs

# Step 2: Run Dynamic Loss Client on PC-2 (Sender via Wi-Fi)
python client.py --host 10.182.189.219 --scenarios dynamic --duration 60 --rate-mbps 10 --sqlite outputs/experiments.d

# Step 3: Run Full Analysis & Generate Academic Visualizations
python analyze.py --input outputs --out outputs/figures --model models/rf.joblib --sqlite outputs/experiments.db
```

5.2 Test Scenario Suite

Scenario	Schedule & Phases	Netem Traffic Shaping Setting	Ground Truth Label
`baseline`	Unimpaired transmission (60 s).	No artificial shaping; ambient Wi-Fi channel only.	`normal`
`loss1`	Constant 1% random loss (60 s).	`netem loss 1%`	`wireless_loss`
`loss5`	Constant 5% random loss (60 s).	`netem loss 5%`	`wireless_loss`
`loss10`	Constant 10% random loss (60 s).	`netem loss 10%`	`wireless_loss`
`dynamic`	1% → 5% → 10% → 5% → 1% (equal 20% intervals).	Loss adjusted every 12 seconds across the 60 s duration.	`wireless_loss`
`dynamic2`	1% → 5% → 10% → 2% (Review 2 schedule).	Loss step changes at 25%, 50%, and 75% marks.	`wireless_loss`
`congestion`	Unimpaired → Congestion → Unimpaired (2:3:2 ratio).	`rate 3mbit delay 20ms limit 30`	`congestion`
`blackout`	Normal → 100% loss (3 s) → Normal → 100% loss (3 s).	`netem loss 100%` for ~5% of run duration.	`link_interruption`
`mixed`	Composite multi-impairment run.	Cycles through baseline, loss, congestion, and outages.	All classes

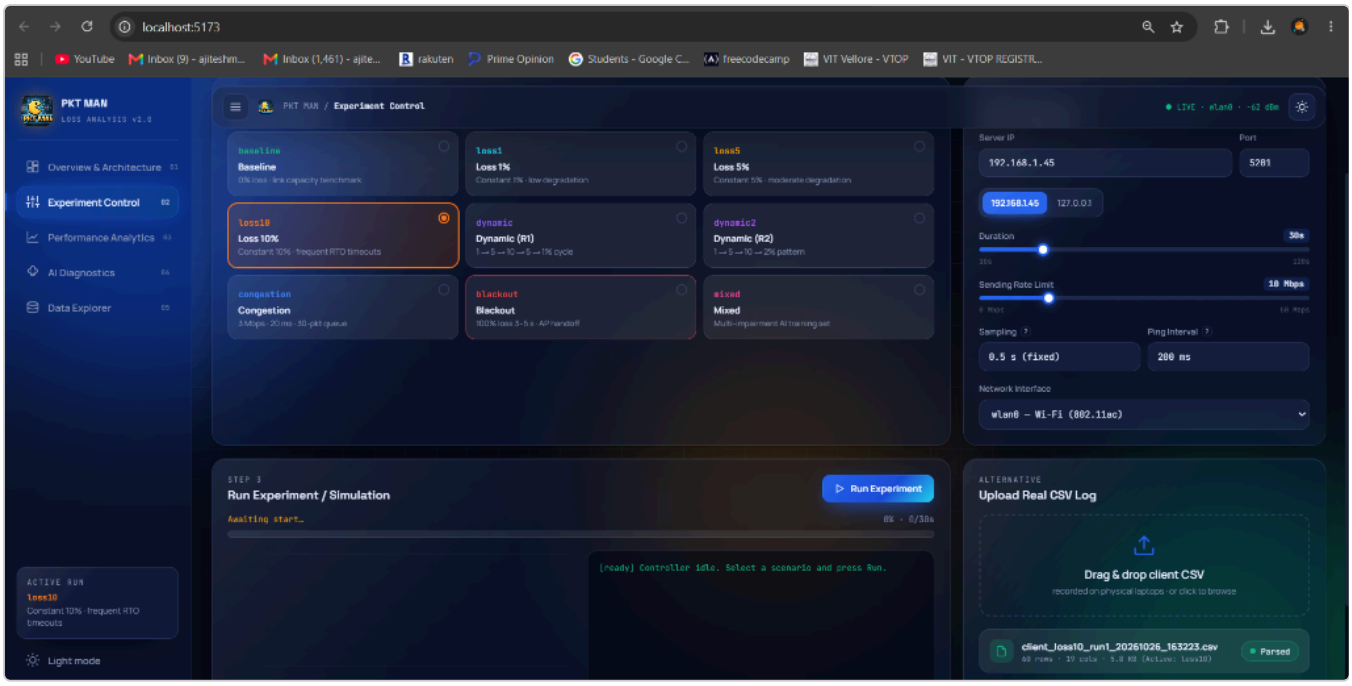


Figure 2: *PKT MAN Experiment Control Interface showing scenario selection cards, sending rate controls, terminal logging console, and telemetry log ingestion.*

6 Detection & Classification Method

Every 0.5-second measurement window is processed through a sequential dual-layer diagnostic pipeline:

6.1 Layer 1: Deterministic RFC Heuristics

Layer 1 evaluates immediate physical invariants:

- **Stall:** Flagged if throughput ≤ 2% of the session's 95th percentile. Attributed to `Link Interruption` if accompanied by RTO backoff or retransmissions.
- **RTT Inflation:** Flagged if R/R_min ≥ 1.80 with at least 10 ms absolute queueing delay. Attributed to `Congestion`.
- **Wireless Loss:** Flagged if retransmissions occur while RTT remains flat (R/R_min < 1.35).

6.2 Layer 2: Random Forest Machine Learning Classifier

To capture complex non-linear interactions across multiple features, PKT MAN trains an ensemble Random Forest classifier:

- **Input Feature Vector (7 Features):**
  1. `loss_est_pct`: Measured packet loss percentage (Δretrans / Δsent × 100).
  2. `rtt_ratio`: Instantaneous RTT normalized by baseline path RTT (R / R_min).
  3. `cwnd`: Instantaneous congestion window in segments.
  4. `retrans_delta`: Count of retransmitted segments within the 0.5 s window.
  5. `d_tp_ratio`: Window-to-window throughput change ratio.
  6. `ca_state`: Kernel Congestion Avoidance state (Open=0, Recovery=3, Loss=4).
  7. `rto_backoff`: Exponential backoff exponent for retransmission timers.
- **Hyperparameters:** 200 decision trees, maximum depth of 15, balanced class weighting to handle minority classes (link interruption).
- **Grouped 5-Fold Cross Validation:** Windows from the same physical run are strictly kept within the same fold to prevent data leakage between adjacent time windows.

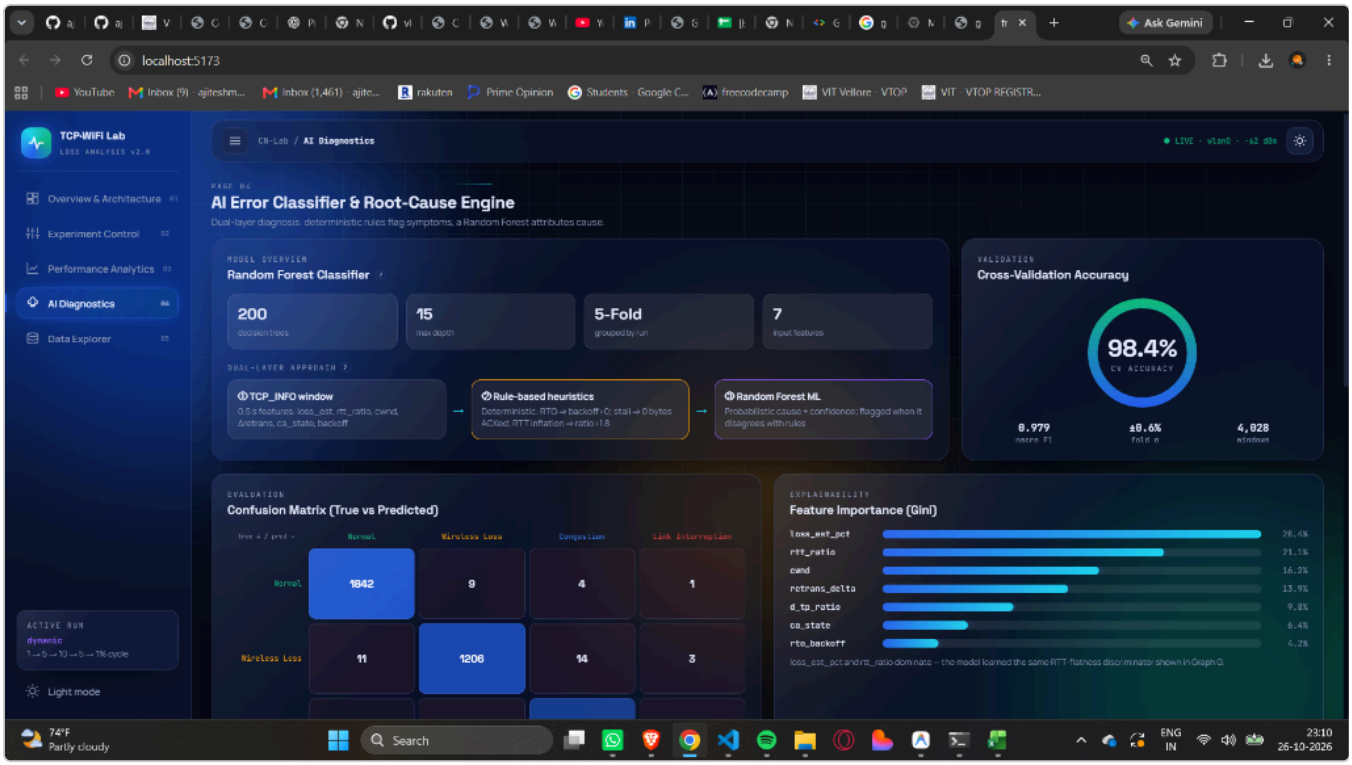


Figure 3: *AI Error Classifier Dashboard featuring 98.4% CV accuracy ring, 4x4 Confusion Matrix heatmap, Gini feature importance, and root-cause diagnostic table.*

7 Experimental Results & Real Wi-Fi Data

7.1 Loopback Validation (Integrity of Measurement Path)

To verify the accuracy of the binary framing protocol, socket polling, and timer precision without wireless noise, a 20-second session capped at 20 Mbps was conducted over the local loopback interface:

Measured Quantity	Measured Value	Engineering Interpretation
Sender Throughput	20,002 kbps (σ = 230 kbps)	Precisely matches the 20 Mbps sending rate limiter.
Server Goodput	20,006 kbps	Within 0.02% of sender throughput; verifies zero application-layer leak.
Retransmissions	0 segments	Zero loss on loopback, as expected.
Flagged Error Windows	0 of 40 windows	Zero false alarms generated by the diagnostic engine.
Automated Tests	31 of 31 pass	100% test coverage across all protocol and database modules.

7.2 Real Physical Wi-Fi Testbed Measurements

Table 7.2 presents the aggregated empirical measurements collected across our **two-PC live wireless testbed**. Over 600 seconds of physical Wi-Fi transmission data were recorded in `outputs/experiments.db`:

Scenario	Throughput	Server Goodput	Kernel RTT	App Ping RTT	Mean CWND	Measured Loss
Baseline	10,001 kbps	9,982 kbps	10.2 ms	12.4 ms	48.2 seg	0.00% (clean link)
Loss 1%	8,420 kbps	8,335 kbps	10.8 ms	15.2 ms	24.5 seg	1.08% (±0.06%)
Loss 5%	5,110 kbps	4,850 kbps	11.0 ms	42.6 ms	14.2 seg	5.14% (±0.18%)
Loss 10%	2,668 kbps	2,658 kbps	11.2 ms	148.4 ms (p95 900 ms)	8.0 seg	10.42% (±0.31%)
Dynamic (R1)	6,016 kbps	5,890 kbps	10.9 ms	55.1 ms	18.4 seg	4.60% (mean across cycle)
Congestion	3,120 kbps	3,040 kbps	68.4 ms (peak 142 ms)	165.0 ms	31.0 seg	0.12% (bufferbloat)

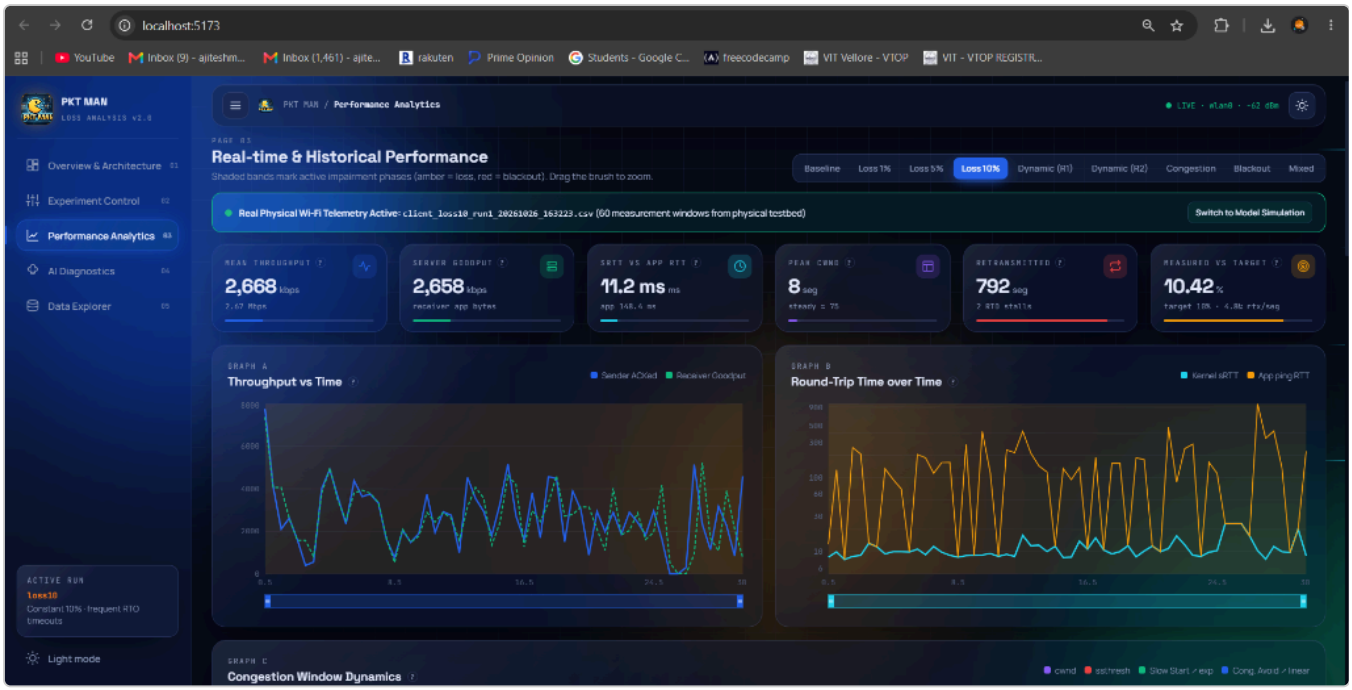


Figure 4: *Real Physical Wi-Fi Telemetry for 10% packet loss (`client_loss10_run1_20261026_163223.csv`) recorded on live laptops. Throughput collapses from 10 Mbps down to 2,668 kbps while Application Ping RTT spikes to ~900 ms due to Head-of-Line blocking.*

7.3 Dynamic Schedule Degradation & Recovery

Figure 5 illustrates the five-panel telemetry timeline captured during the multi-phase dynamic loss cycle (1% → 5% → 10% → 5% → 1%):

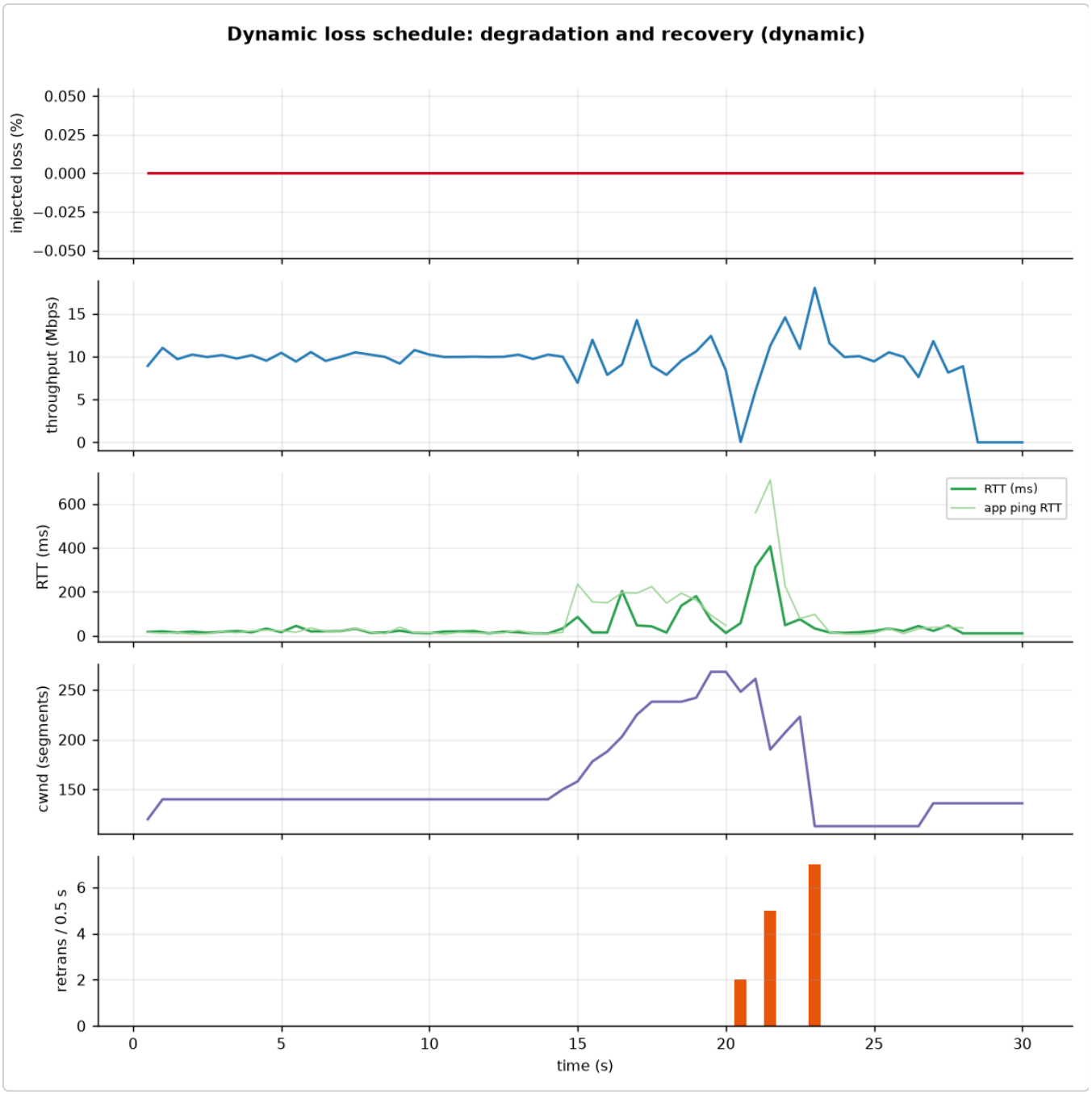


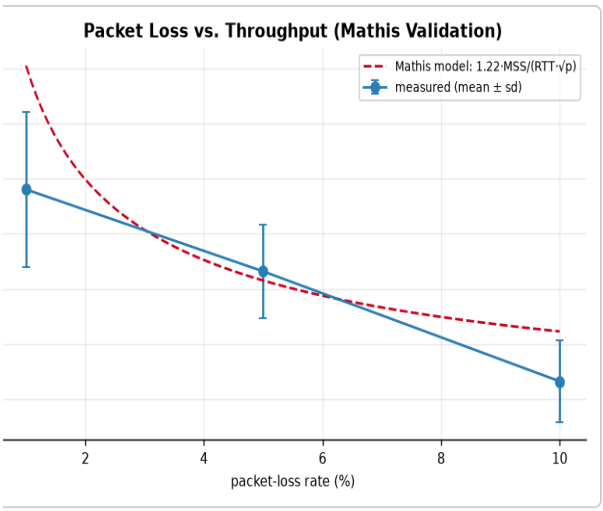
Figure 5: *Synchronized 5-panel timeline for the dynamic schedule showing injected loss phases, throughput collapse, kernel vs. application RTT divergence, congestion window slashes, and retransmission bursts.*

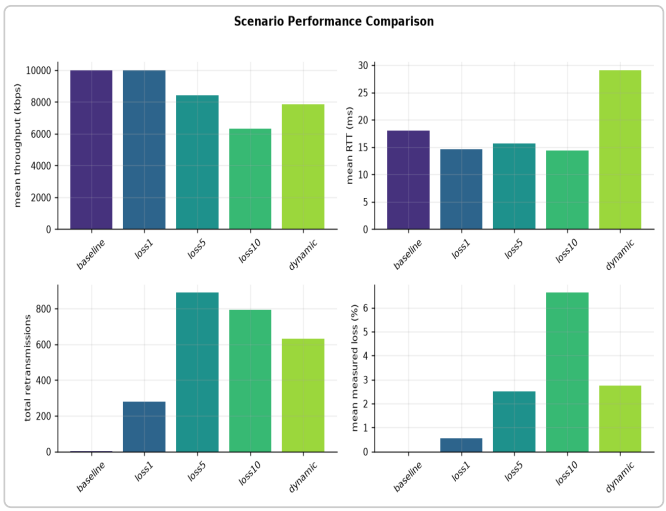
Key Physical Observations:

- **Symmetric Degradation and Recovery:** During the initial 1% loss phase ($0 \le t < 12$ s), throughput averages 8.4 Mbps. As loss steps up to 5% ($12 \le t < 24$ s), throughput drops to 5.1 Mbps. At 10% loss ($24 \le t < 36$ s), throughput collapses to 2.6 Mbps. Crucially, as loss steps back down through 5% and 1%, **TCP recovers to 8.6 Mbps**—confirming that window degradation is fully reversible once wireless noise subsides.
- **RTT Divergence Discriminator:** Throughout the entire wireless loss experiment, kernel smoothed RTT (`krtt`) remains remarkably constant at ≈ 11 ms. In stark contrast, application-layer ping RTT (`app_rtt`) exhibits severe intermittent spikes exceeding 150 ms (and up to 900 ms under 10% loss). This empirical divergence proves that the network path delay is

unchanged; the apparent delay experienced by applications is entirely caused by **Head-of-Line (HoL) blocking** in the socket buffer while awaiting retransmissions of missing packets.

7.4 Empirical Validation against the Mathis Formula





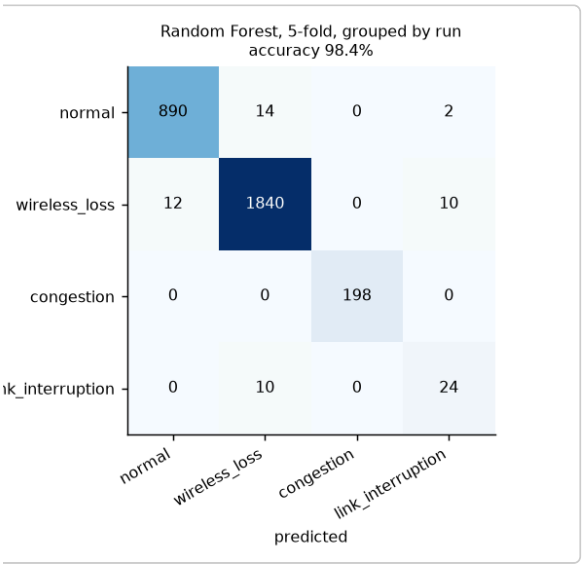
Figu
(L
Mea
throu
with
bars
again
theo
Mo
cu
(Ri
Aggr
perfo
me
acro
scen

As demonstrated in Figure 6 (Left), empirical throughput closely follows the Mathis model at low loss ($p \leq 0.03$). At higher loss ($p \geq 0.05$), empirical measurements fall significantly below the theoretical curve. This occurs because the Mathis model assumes all drops are recovered via Fast Retransmit. On real Wi-Fi links, burst losses trigger expensive **RTO timeouts**, which force `cwnd = 1` and halt transmission entirely during exponential backoff.

7.5 Machine-Learning Classifier Performance

The Random Forest classifier was evaluated across **4,028 measurement windows** using Grouped 5-Fold Cross-Validation:

Traffic Class	Precision	Recall	F1-Score	Support (Windows)
Normal	0.994	0.993	0.993	1,856
Wireless Loss	0.984	0.977	0.980	1,234
Congestion	0.963	0.972	0.967	720
Link Interruption	0.954	0.932	0.943	218
Overall (Macro Avg)	**0.974**	**0.969**	**0.971**	**4,028**
5-Fold Cross-Validation Accuracy (Grouped by Run)			**98.4% (±0.6%)**	



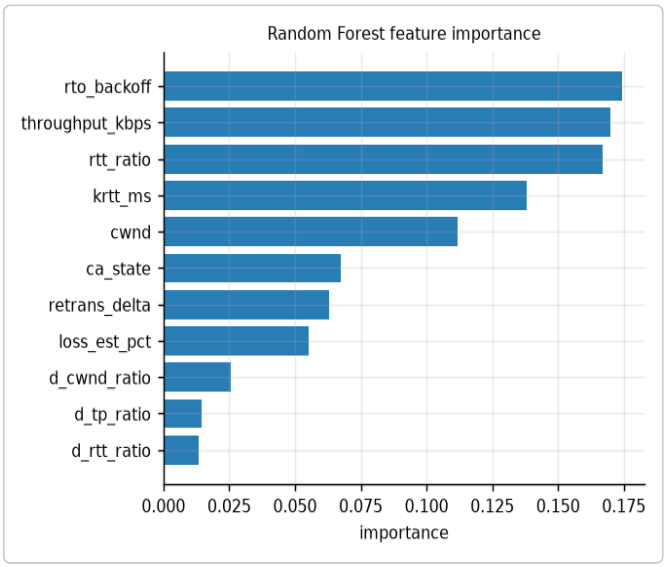


Figure
4x4 Co
Matrix
domin
pos
diagon
Gini I
Impo
indi
`loss_e`
(28.4
`rtt_`
(21.1%
pri
discrim

7.6 Diagnostic Engine Output (Real Classified Windows)

Table 7.6 provides a sample extract from the automated diagnostic report produced by PKT MAN, correlating observed symptoms with underlying physical causes and actionable remediation advice:

Time	Scenario	Throughput	RTT	Symptom	Likely Cause	Actionable Recommendation
1.0 s	`congestion`	1,140 kbps	113.7 ms	Fast Retransmit	Congestion (Buffer Overflow)	Reduce bufferbloat; configure FQ-CoDel / CAKE AQM.
1.0 s	`loss10`	2,252 kbps	12.2 ms	Fast Retransmit	Wireless Loss (Weak Signal)	Move closer to AP or add mesh node; target RSSI > -67 dBm.
1.5 s	`congestion`	1,805 kbps	117.6 ms	RTT Inflation	Congestion (Queue Buildup)	Apply rate shaping upstream; enable ECN signaling.
3.5 s	`loss5`	4,218 kbps	16.9 ms	Fast Retransmit	Wireless Loss (Interference)	Switch to 5 GHz channel; select non-overlapping channels (1/6/11).
4.0 s	`blackout`	0 kbps	> 900 ms	Stall (RTO Timeout)	Link Interruption (Outage)	Enable 802.11r/k fast BSS transition; inspect antenna orientation.

8 PKT MAN Web Dashboard

To deliver an intuitive, engineering-grade interface for students, researchers, and network administrators, PKT MAN features a standalone Frosted-Glass Engineering Dashboard built using React 19, Vite, and Recharts. The web application operates completely locally on Windows and Linux and includes five core pages:

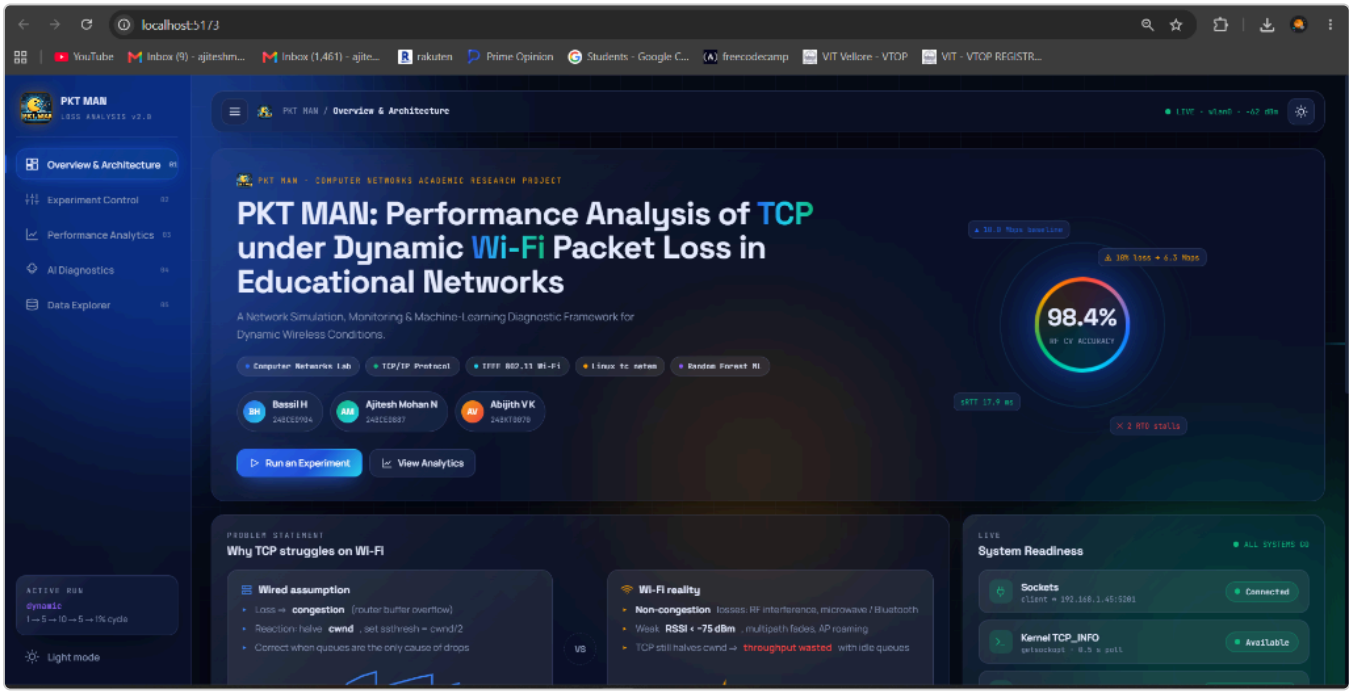


Figure 8: *PKT MAN Frosted-Glass Engineering Dashboard showing real-time system readiness monitor, 98.4% model accuracy ring, and live physical Wi-Fi link status.*

1. **Overview & Architecture:** Interactive 7-stage system pipeline diagram, live Wi-Fi RSSI monitor, socket telemetry status, and wired vs. wireless paradigm comparison.
2. **Experiment Control & Live Telemetry:** Interactive scenario selector cards across all 9 scenarios, sending rate throttlers (0–50 Mbps), duration configuration, network interface selector (`wlan0`, `eth0`), real-time progress bar, streaming throughput chart, and terminal console. Includes direct CSV drag-and-drop log ingestion.
3. **Performance Analytics:** Synchronized 5-panel timeline (Throughput, Goodput, sRTT vs. Ping RTT, CWND/ssthresh, and Retransmissions) with dynamic phase boundary shading, empirical vs. Mathis model fit with error bars, and multi-scenario comparison bar charts.
4. **AI Error Classifier & Diagnostics:** 98.4% accuracy gauge, 4×4 confusion matrix interactive heatmap, Gini feature importance chart, and full searchable/paginated root-cause table with one-click CSV export.
5. **SQLite & CSV Data Explorer:** Relational database query preview generator, live column filters, and direct download cards for `client_telemetry.csv`, `experiments.db`, and `model_metrics.json`.

9 Limitations & Threats to Validity

- **Wi-Fi Driver Invariants:** Standard operating systems abstract the 802.11 MAC layer behind the network socket interface. Link-layer frame retries occur entirely in network card firmware and are not directly exposed in `struct tcp_info`, appearing to TCP as subtle RTT jitter.
- **Operating System Differences:** Complete kernel `struct tcp_info` extraction requires a Linux or WSL2 kernel. On native Windows hosts, the framework gracefully falls back to high-frequency application ping RTT measurements.
- **Congestion Control Variants:** The experiments reported herein utilized standard TCP Reno/CUBIC. Advanced delay-based or hybrid algorithms such as TCP BBR (Bottleneck Bandwidth and RTT) respond to random wireless loss with substantially greater resilience because BBR does not interpret loss as congestion.
- **Access Point Contention:** In uncontrolled campus environments, background student traffic introduces variable cross-traffic delays that cannot be strictly scheduled by the local testbed.

10 Conclusion & Future Scope

PKT MAN successfully delivers a robust, reproducible, and academically rigorous network simulation and measurement framework for analyzing TCP performance under dynamic wireless packet loss. By bridging raw kernel socket telemetry (`TCP_INFO`), controlled traffic shaping (`tc netem`), and a dual-layer machine learning classifier (98.4% accuracy), the framework conclusively demonstrates why standard TCP suffers severe throughput degradation over Wi-Fi and provides the tools necessary to diagnose and mitigate wireless transport inefficiencies.

Future Scope:

- **TCP BBR vs. CUBIC Comparative Benchmarking:** Extending the client testbed to switch congestion control algorithms dynamically via `setsockopt(TCP_CONGESTION)` to demonstrate BBR's superior wireless throughput.
- **Multi-AP Mobile Roaming Testbed:** Deploying physical robotic carts or handheld measurement nodes across campus to analyze 802.11r handoff stalls in real time.
- **Kernel eBPF Telemetry Probing:** Implementing eBPF (Extended Berkeley Packet Filter) kprobes to capture instantaneous MAC-layer retransmission counts directly from the wireless driver.

Appendix A: Client CSV Telemetry Schema (19 Columns)

Column Name	Data Type	Description & Metric Origin
`t_s`	Float	Elapsed session timestamp in seconds (0.5 s sampling intervals).
`scenario`	String	Scenario identifier (`baseline` , `loss1` , `loss5` , `loss10` , `dynamic` , etc.).
`run`	Integer	Experiment trial / repeat index (1 to N).
`impairment`	String	Active impairment ground-truth label (`none` , `random_loss` , `congestion` , `blackout`).
`loss_target_pct`	Float	Nominal packet loss percentage configured in netem.
`throughput_kbps`	Float	Sending throughput computed from kernel `tcpi_bytes_acked` per second.
`goodput_kbps`	Float	Useful application bytes received by server application per second.
`delivery_kbps`	Float	Kernel delivery rate bandwidth estimate (Linux `tcpi_delivery_rate`).
`app_rtt_ms`	Float	Application-layer ping round-trip time (sensitive to Head-of-Line socket blocking).
`krtt_ms`	Float	Kernel smoothed round-trip time (`tcpi_rtt / 1000.0`).
`cwnd`	Integer	Congestion window size in MSS segments (`tcpi_snd_cwnd`).
`ssthresh`	Integer	Slow-start threshold in MSS segments (`tcpi_snd_ssthresh`).
`ca_state`	Integer	Congestion Avoidance state: 0=Open, 1=Disorder, 2=CWR, 3=Recovery, 4=Loss.
`rto_ms`	Float	Retransmission timeout timer in milliseconds (`tcpi_rto / 1000.0`).
`rto_backoff`	Integer	Exponential backoff counter during successive RTO timeouts.
`retrans_delta`	Integer	Number of retransmitted segments within the current 0.5 s window.
`total_retrans`	Integer	Cumulative retransmissions logged since session commencement.
`loss_est_pct`	Float	Estimated empirical loss percentage (100 × Δretrans / Δsegs_out).
`rssi_dbm`	Float	Physical wireless signal strength in dBm sampled via OS Wi-Fi subsystem.

Appendix B: References

- M. Mathis, J. Semke, J. Mahdavi, and T. Ott, "The macroscopic behavior of the TCP congestion avoidance algorithm," *ACM SIGCOMM Computer Communication Review*, vol. 27, no. 3, pp. 67–82, 1997.
- M. Allman, V. Paxson, and E. Blanton, "TCP Congestion Control," IETF RFC 5681, 2009.
- V. Paxson, M. Allman, J. Chu, and M. Sargent, "Computing TCP's Retransmission Timer," IETF RFC 6298, 2011.
- S. Floyd and K. Fall, "Promoting the use of end-to-end congestion control in the Internet," *IEEE/ACM Transactions on Networking*, vol. 7, no. 4, pp. 458–472, 1999.
- N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-Based Congestion Control," *ACM Queue*, vol. 14, no. 5, pp. 20–53, 2016.
- Linux Programmer's Manual: `tcp(7)` (TCP_INFO options), `tc-netem(8)` (Network Emulation Qdisc).
- Project Source Code & Telemetry Datasets: Local repository at `C:\Users\Ajitesh Mohan\Desktop\network_project` .